

Contents

Guia: Replicando Token Economy via Git Submodules	1
1. Arquitetura: O que é Compartilhável vs. Específico	2
2. Criando o Repositório Compartilhado	2
3. Adicionando o Submodule em Projetos Existentes	4
4. Criando Projeto Novo com Token Economy	5
5. Sincronizando Atualizações	5
6. Submodule vs. Alternativas	6
7. Melhores Práticas	6
8. Caso Real Aplicado: <code>kit-fundacao-aidd</code>	7
9. Caso Real Aplicado: <code>code-review-graph</code>	8
10. Caso Real Aplicado: <code>impeccable</code>	9
11. Exemplo Prático: Fábrica → Novo Projeto	10
12. Conclusão	10

Guia: Replicando Token Economy via Git Submodules

Data: 2026-08-07 (atualizado em 2026-08-12 — seções 8-10: casos reais aplicados) **Objetivo:** Reutilizar skills de compressão, MCPs, hooks, scripts e configs de economia de tokens em qualquer projeto novo ou existente.

As seções 2 a 7 e 11-12 descrevem o padrão em abstrato (repositório hipotético token-economy-shared, placeholders seu-usuario). As seções 8-10 documentam os 3 submodules **reais**, já criados e em produção nesta fábrica — `kit-fundacao-aidd`, `code-review-graph` e `impeccable` — cada um com seu próprio fluxo de instalação real (nem todo submodule é “código que roda”; nos dois últimos, o submodule é só referência, a ferramenta real vem de PyPI/npm).

1. Arquitetura: O que é Compartilhável vs. Específico

Camada Global (Submodule) – Reutilizável em QUALQUER projeto

Componente	Caminho	Função
lean-ctx	.claude/skills/lean-ctx/	Economia de contexto: grep antes de read
headroom	.claude/skills/headroom/	Compressão de logs > 7 linhas
caveman	.claude/skills/caveman/	Respostas telegráficas, sem enrolação
rtk-memory	.claude/skills/rtk-memory/	Registro de erros e padrões
pre-flight-check	.claude/skills/pre-flight-check/	Validação antes de commit/deploy
calcular-gastos-sessao	.claude/skills/calcular-gastos-sessao/	Cálculo de gastos por sessão
fable-method	.claude/skills/fable-method/	Loop de resolução de problemas
fable-judge	.claude/skills/fable-judge/	Verificação adversarial de trabalho
self-learning	.claude/skills/self-learning/	Captura de golden paths
code-review-graph	.code-review-graph/	Grafo de conhecimento do codebase
pytest.ini	pytest.ini	Configuração de testes
requirements.txt	requirements.txt	Dependências Python

Camada de Projeto (Fica no repositório do projeto)

Componente	Caminho	Função
Skills do domínio	.claude/skills/<dominio>/	Skills específicas do projeto
Agents	.claude/agents/	Subagentes do projeto
Commands	.claude/commands/	Comandos do projeto
Specs	SPEC*.md	Especificações do projeto
Scripts	scripts/	Scripts determinísticos
Templates	templates/	Templates de output
CLAUDE.md	CLAUDE.md	Regras do projeto

2. Criando o Repositório Compartilhado

2.1 Estrutura do Repositório token-economy-shared

```
token-economy-shared/  
├── .claude/  
│   └── skills/  
│       └── lean-ctx/
```

```

|       |   └─ SKILL.md
|       |   └─ headroom/
|       |       └─ SKILL.md
|       |   └─ caveman/
|       |       └─ SKILL.md
|       |   └─ rtk-memory/
|       |       └─ SKILL.md
|       |   └─ pre-flight-check/
|       |       └─ SKILL.md
|       |   └─ calcular-gastos-sessao/
|       |       └─ SKILL.md
|       |   └─ fable-method/
|       |       └─ SKILL.md
|       |   └─ fable-judge/
|       |       └─ SKILL.md
|       |   └─ self-learning/
|       |       └─ SKILL.md
|   └─ .code-review-graph/
|       └─ .gitignore
|   └─ pytest.ini
|   └─ requirements.txt
|   └─ setup-links.sh
|   └─ setup-links.ps1
|   └─ README.md

```

2.2 Comandos para Criar

1. Criar o repositório compartilhado

```

mkdir token-economy-shared
cd token-economy-shared
git init

```

2. Copiar as skills globais do projeto atual

```

cp -r ../proj_fabrica-de-livros/.claude/skills/lean-ctx .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/headroom .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/caveman .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/rtk-memory .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/pre-flight-check .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/calcular-gastos-sessao .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/fable-method .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/fable-judge .claude/skills/
cp -r ../proj_fabrica-de-livros/.claude/skills/self-learning .claude/skills/

```

3. Copiar configs

```

cp ../proj_fabrica-de-livros/pytest.ini .
cp ../proj_fabrica-de-livros/requirements.txt .
cp ../proj_fabrica-de-livros/scripts/setup-links.sh .
cp ../proj_fabrica-de-livros/scripts/setup-links.ps1 .

```

4. Criar .gitignore

```

echo ".code-review-graph/" > .gitignore
echo "__pycache__/" >> .gitignore
echo "*.pyc" >> .gitignore

```

5. Commit e push

```

git add .
git commit -m "feat: token economy shared infrastructure"
git remote add origin git@github.com:seu-usuario/token-economy-shared.git
git push -u origin main

```

3. Adicionando o Submodule em Projetos Existentes

3.1 Comando Único

```
# No diretório do projeto
git submodule add git@github.com:seu-usuario/token-economy-shared.git .token-economy
```

3.2 Estrutura Resultante

```
meu-projeto/
├── .token-economy/           ← SUBMODULE (read-only)
│   ├── .claude/skills/
│   │   ├── lean-ctx/
│   │   ├── headroom/
│   │   ├── caveman/
│   │   └── ...
│   ├── pytest.ini
│   └── requirements.txt
├── .claude/                  ← PROJETO (read-write)
│   ├── skills/
│   │   └── minha-skill/      ← Skills específicas do projeto
│   ├── agents/
│   └── commands/
├── CLAUDE.md
└── scripts/
```

3.3 Script de Setup Automatizado

Criar `setup-token-economy.sh` no projeto:

```
#!/bin/bash
# Setup Token Economy via Submodule

SUBMODULE_PATH=".token-economy"

# 1. Adicionar submodule (se não existe)
if [ ! -d "$SUBMODULE_PATH" ]; then
    git submodule add git@github.com:seu-usuario/token-economy-shared.git
    $SUBMODULE_PATH
fi

# 2. Atualizar submodule
git submodule update --init --recursive

# 3. Criar junctions (Windows) ou symlinks (macOS/Linux)
if [[ "$OSTYPE" == "msys" || "$OSTYPE" == "cygwin" ]]; then
    # Windows: junctions
    cmd //c "mklink /J .claude\skills\lean-ctx $SUBMODULE_PATH\.claude\skills\lean-ctx"
    cmd //c "mklink /J .claude\skills\headroom $SUBMODULE_PATH\.claude\skills\headroom"
    cmd //c "mklink /J .claude\skills\caveman $SUBMODULE_PATH\.claude\skills\caveman"
    # ... mais junctions
else
    # macOS/Linux: symlinks
    ln -s "$SUBMODULE_PATH/.claude/skills/lean-ctx" ".claude/skills/lean-ctx"
    ln -s "$SUBMODULE_PATH/.claude/skills/headroom" ".claude/skills/headroom"
    ln -s "$SUBMODULE_PATH/.claude/skills/caveman" ".claude/skills/caveman"
    # ... mais symlinks
fi

# 4. Copiar configs (se não existem)
```

```
[ ! -f pytest.ini ] && cp $SUBMODULE_PATH/pytest.ini .
[ ! -f requirements.txt ] && cp $SUBMODULE_PATH/requirements.txt .

# 5. Instalar dependências
pip install -r requirements.txt

echo "Token Economy configurado com sucesso!"
```

4. Criando Projeto Novo com Token Economy

4.1 Template de Projeto

```
# 1. Criar projeto a partir do template
git clone git@github.com:seu-usuario/template-projeto.git meu-projeto
cd meu-projeto

# 2. Renomear
git remote set-url origin git@github.com:seu-usuario/meu-projeto.git

# 3. O submodule já vem configurado no template
git submodule update --init --recursive

# 4. Criar junctions/symlinks
bash setup-token-economy.sh

# 5. Personalizar CLAUDE.md
# Editar CLAUDE.md com as regras específicas do projeto
```

4.2 Template CLAUDE.md Mínimo

```
# Meu Projeto

## Regras
- PT-BR estrito
- Sem metatexto nos artefatos
- Auto-correção interna antes de entrega

## Stack
- Python 3.11+
- pytest para testes
- Token Economy via `token-economy/`

## Comandos
- `/executar` – Roda o pipeline principal
- `/validar` – Roda testes e lint
```

5. Sincronizando Atualizações

5.1 Atualizar o Submodule no Projeto

```
# Atualizar para a última versão
cd token-economy
git pull origin main
cd ..

# Commitar a mudança
git add token-economy
git commit -m "chore: update token-economy to latest"
git push
```

5.2 Propagar Atualização para Todos os Projetos

```
# Em cada projeto que usa o submodule
git submodule update --remote .token-economy
git add .token-economy
git commit -m "chore: sync token-economy"
git push
```

5.3 Script de Sincronização em Massa

```
#!/bin/bash
# sync-all-projects.sh
PROJECTS=(
    "/caminho/projeto1"
    "/caminho/projeto2"
    "/caminho/projeto3"
)

for proj in "${PROJECTS[@]}; do
    echo "=== Sincronizando $proj ==="
    cd "$proj"
    git submodule update --remote .token-economy
    git add .token-economy
    git commit -m "chore: sync token-economy" || echo "Nada para commitar"
    git push || echo "Erro no push"
    cd -
done
```

6. Submodule vs. Alternativas

6.1 Git Submodule (Recomendado)

Prós: - Versionado junto com o projeto - Atualização controlada (`git submodule update`) - Funciona offline após clone - Integração nativa com git

Contras: - Requer `git submodule update --init` após clone - Junctions/symlinks precisam ser recriados manualmente

6.2 Git Subtree

Prós: - Mais simples que submodule - Código fica integrado no repositório

Contras: - Duplica o código (não é referência) - Atualização mais trabalhosa

6.3 Package Manager (pip/npm)

Prós: - Instalação automatizada - Versionamento semântico

Contras: - Não funciona bem com arquivos `.md` e configs - Requer registro em registry público/privado

6.4 Copiar/Colar (Não Recomendado)

Prós: - Simples

Contras: - Sem sincronização - Duplicação massiva - Manutenção impossível

7. Melhores Práticas

1. **Versão do Submodule:** usar tag semântica (`v1.0.0`) no repositório compartilhado
2. **Documentação:** manter `README.md` atualizado no repositório compartilhado
3. **Testes:** rodar testes no repositório compartilhado antes de publicar
4. **Backup:** sempre ter pelo menos 2 cópias do repositório compartilhado
5. **Branches:** usar `main` para versão estável e `dev` para experimentação

8. Caso Real Aplicado: kit-fundacao-aidd

Diferente dos exemplos anteriores (hipotéticos, com placeholders seu-usuario), este é um submodule **real**: criado, testado (21/21 testes próprios + validado sem efeitos colaterais nesta fábrica, 665/665) e registrado em tooling/kit-fundacao-aidd, apontando para o repositório privado github.com/Heverton-web/kit-fundacao-aidd.

Ele não replica token economy — replica 5 decisões de **engenharia de software** já maduras nesta fábrica (origem: docs/manual-replicar-praticas-acima-media.md), generalizadas para qualquer projeto. É citado aqui porque é a aplicação mais recente e mais completa do padrão “submodule reutilizável” descrito neste guia.

8.1 O que o pacote empacota

Peça	Caminho no submodule	Função
Builder ≠ Critic	agents/ builder.md.template, agents/critic.md.template	quem gera nunca é quem aprova; critic com tools: Read travado
Crítico determinístico	scripts/validar_template.py.template	aqueleto de gate por regex/contagem — nunca LLM para checagem de formato
Registro declarativo	scripts/registro_declarativo_scaffold.py	gera TIPOS = {...} + dispatch (princípio Aberto/Fechado)
Nunca commitar vermelho	hooks/pre-commit.template	bloqueia commit se a suíte falhar — gate mecânico, não promessa
Postmortem que vira teste	templates/POSTMORTEM.md, scripts/ postmortem_para_teste.py	toda linha de “Prevenção” nasce com um stub de teste de regressão
Diagnóstico	analisar-projeto.py	só leitura — detecta stack, convenção de agentes, hook existente, candidatos a registro declarativo
Instalador	instalar.py	aplica as peças escolhidas
Skill interativa	skills/kit-fundacao-aidd/ SKILL.md	versão conversacional do instalador para uso no Claude Code

8.2 A diferença chave: o instalador nunca é destrutivo

O setup-token-economy.sh hipotético da seção 3.3 faz cp/mklink direto, só checando “não existe ainda” para 2 arquivos de config. O instalar.py do kit-fundacao-aidd vai além, por exigência explícita registrada em melhorias/2026-08-12-kit-fundacao-aidd.md:

1. **Nunca remove ou sobrescreve** nada já configurado no projeto-alvo. Em colisão: mescla não-destrutiva (o hook de pre-commit ganha um bloco extra no final, nunca é substituído) ou se recusa a escrever e explica o motivo.
2. **Sempre explica antes de aplicar** — cada ação de escrita imprime o que vai ser criado/alterado e por quê, além do preview do conteúdo.
3. **Dry-run por padrão** — só grava com a flag --aplicar explícita.

4. **Idempotente** — peça já presente é reportada como “já presente, nada a fazer”, nunca como “corrigida”.

8.3 Uso em qualquer projeto (novo ou existente)

```
# 1. Registrar o submodule
git submodule add https://github.com/Heverton-web/kit-fundacao-aidd.git tooling/kit-fundacao-aidd
git submodule update --init --recursive

# 2. Diagnóstico — só leitura, nunca grava
python tooling/kit-fundacao-aidd/analisar-projeto.py .

# 3. Ver o plano (dry-run — nada é gravado ainda)
python tooling/kit-fundacao-aidd/instalar.py . --peca todas

# 4. Só depois de revisar o plano, aplicar de fato
python tooling/kit-fundacao-aidd/instalar.py . --peca todas --aplicar

--peca aceita builder-critic, gate, registro, hook, postmortem ou todas — permite instalar uma
peça por vez em vez de tudo de uma vez.
```

8.4 Atualizando o submodule

```
git -C tooling/kit-fundacao-aidd pull origin main
git add tooling/kit-fundacao-aidd
git commit -m "chore: sync kit-fundacao-aidd"
git push
```

8.5 Documentação de origem

- melhorias/2026-08-12-kit-fundacao-aidd.md — plano de arquitetura
- relatorios/2026-08-12-kit-fundacao-aidd.md/.pdf — relatório da sessão que criou o pacote (testes, commits, validações)
- docs/manual-replicar-praticas-acima-media.md — as 5 práticas na forma original, ainda acopladas ao vocabulário desta fábrica

9. Caso Real Aplicado: code-review-graph

Diferente do kit-fundacao-aidd, este submodule vendoriza só o **código-fonte para referência/leitura** — a ferramenta que roda de fato vem do PyPI, não do submodule (CLAUDE.md, seção 6).

9.1 Instalar a ferramenta (PyPI, não o submodule)

```
pip install code-review-graph      # ou: pipx install code-review-graph
```

9.2 Configurar automaticamente para a IDE/CLI detectada

```
code-review-graph install
```

Escreve/atualiza o .mcp.json do projeto. Manualmente, a entrada é:

```
"code-review-graph": {
  "command": "uvx",
  "args": ["code-review-graph", "serve"],
  "cwd": "<caminho-absoluto-do-projeto>",
  "type": "stdio"
}
```

(uvx exige a ferramenta uv instalada — roda a partir do PyPI sem instalação persistente.)

9.3 Build inicial do grafo

```
code-review-graph build
```


9.4 Vendorizar o código-fonte como submodule (opcional, só referência)

```
git submodule add https://github.com/tirth8205/code-review-graph.git .claude/mcp-servers/code-review-graph
git submodule update --init --recursive
```

9.5 Manter o grafo sempre atualizado (opcional, mecânico)

No hook de pre-commit, com `|| true` para nunca bloquear commit por falha de ambiente:

```
code-review-graph update || true
code-review-graph detect-changes --brief || true
```

9.6 Reiniciar e verificar

Reiniciar o Claude Code para carregar o novo MCP server, depois:

```
code-review-graph status
```

ou usar as tools MCP (`list_graph_stats_tool`, `get_architecture_overview_tool` etc.) dentro do próprio Claude Code.

10. Caso Real Aplicado: impeccable

Fluxo oficial documentado pelo próprio pacote (`README.md` do repositório) — mais simples que replicar a vendorização manual.

10.1 Instalar a partir da raiz do projeto-alvo

```
npx impeccable install
```

Detecta a ferramenta (Claude Code, Cursor, Windsurf etc.) e escreve o skill + hooks nativos daquela plataforma automaticamente. Preserva hooks/configs já existentes — não sobrescreve entradas não relacionadas.

10.2 Inicializar o contexto de design do projeto

Dentro do Claude Code (ou equivalente):

```
/impeccable init
```

Gera `PRODUCT.md` (e oferece `DESIGN.md`) com público, marca, voz, cores, tipografia e componentes — contexto que os comandos seguintes usam.

10.3 Vendorizar o código-fonte como submodule (opcional, mesmo padrão do code-review-graph)

Foi o que foi feito nesta fábrica:

```
git submodule add https://github.com/pbakaous/impeccable.git .claude/skills/impeccable
git submodule update --init --recursive
```

Guarda uma cópia read-only do repositório para referência/auditoria — **não substitui** o passo 10.1 (`npx impeccable install`), que é o que de fato ativa o skill/hooks na ferramenta.

10.4 Usar

```
/impeccable detect src/          # varre um diretório por antipadrões de design
/impeccable audit                # auditoria completa
```

ou os comandos equivalentes documentados em `reference/*.md` do pacote (`craft`, `polish`, `harden`, `document` etc.).

10.5 Manter atualizado

```
npx impeccable update
```

11. Exemplo Prático: Fabrica → Novo Projeto

```
# 1. Criar novo projeto
mkdir projeto-novo && cd projeto-novo
git init

# 2. Adicionar token economy
git submodule add git@github.com:seu-usuario/token-economy-shared.git .token-economy

# 3. Setup
bash .token-economy/setup-token-economy.sh

# 4. Criar CLAUDE.md mínimo
cat > CLAUDE.md << 'EOF'
# Projeto Novo
## Regras
- PT-BR estrito
- Token Economy ativo
EOF

# 5. Commit inicial
git add .
git commit -m "feat: initial setup with token economy"
git remote add origin git@github.com:seu-usuario/projeto-novo.git
git push -u origin main
```

12. Conclusão

Usar Git Submodules para compartilhar a infraestrutura de Token Economy é a abordagem mais escalável e sustentável. Permite:

- **Replicação instantânea** em novos projetos
- **Atualização centralizada** que propaga para todos os projetos
- **Manutenção reduzida** (atualizar uma vez, propagar para N projetos)
- **Consistência** entre projetos da mesma organização

A separação entre Camada Global (submodule) e Camada de Projeto (repositório) garante flexibilidade sem perda de controle.